ServiNow Informe técnico y guía de estudio

Estado del código al 30 de septiembre de 2026

Propósito del documento

Explicar qué está programado, cómo funciona y cómo presentarlo. El informe describe el proyecto Android real, no solo los mockups. Incluye los flujos desarrollados hasta la vista 20, sus clases, estados, validaciones, mapas, fotos y almacenamiento.

Conclusión principal

ServiNow es una aplicación Android en Kotlin y Jetpack Compose con formularios, navegación, mapas interactivos y operaciones locales. Ya permite crear, editar y eliminar publicaciones locales, registrar comentarios y marcar interés. No es todavía una plataforma conectada entre usuarios: faltan autenticación real, base de datos remota, subida de imágenes y moderación real.

Proyecto inspeccionado

/home/fafa/Documents/Codex/2026-09-21/bue/outputs/ServiNow

Paquete: com.example.servinow. Punto de entrada: MainActivity.kt. Versión de la app: 1.0.

Cómo leerlo

Primero estudia el estado funcional y la arquitectura. Después sigue los ejemplos de flujo, los modelos de datos y los métodos. Las últimas páginas contienen pruebas, limitaciones, un guion para exponer y un índice del código para localizar cada implementación.

Este informe documenta el estado observado; no certifica que los requisitos completos de la asignatura ya estén terminados.

1. Qué está programado

Módulo	Comportamiento actual
Login	Campos, ojo para mostrar/ocultar, Recordarme, validación, botón Google y navegación. No autentica.
Registro y dirección	Valida nombre, correo, contraseña y confirmación. Abre mapa y permite confirmar coordenadas en memoria. No crea cuenta.
Recuperación 05-07	Correo → aviso de revisión de correo → nueva contraseña. Flujo de demostración sin envío ni token real.
Explorar 08-10	Feed de tres trabajos de ejemplo, búsqueda, categorías, filtros, orden y mapa con marcadores.
Detalle y comentarios 11-12	Detalle, ubicación, interés reversible y comentarios por trabajo con persistencia local.
Crear 13-17	Categoría/título → descripción/presupuesto/fotos → punto en mapa → revisión → publicación pendiente local.
Gestionar 18-20	Listado propio con filtros, editar, confirmar eliminación, motivo de rechazo de ejemplo y corrección.

La entrada “Explorar vistas de demostración” permite acceder al contenido sin fingir un inicio de sesión exitoso. Los trabajos del feed no proceden de una API: están en exampleJobs.

Acciones que aún son avisos

Google, inicio de sesión real, envío de recuperación, avisos generales y cuenta no están conectados a servicios. Las ofertas, perfiles/CV, reputación, logros, roles efectivos y moderación requieren desarrollo posterior. En las publicaciones propias no hay una acción real de moderador que cambie Pendiente a Verificada.

2. Arquitectura real del proyecto

La organización es por funcionalidad, con un enfoque MVVM sencillo y flujo unidireccional de datos. No es una Clean Architecture completa: no existen capas implementadas de repositorios, casos de uso y fuentes remotas independientes.

Parte	Responsabilidad
View: Compose	Funciones @Composable dibujan pantallas y emiten eventos mediante callbacks.
ViewModel	Recibe acciones, valida, modifica el estado y, actualmente, también lee/escribe datos locales.
Model / UiState	data class describe la información de negocio y el estado que debe mostrar la pantalla.
Persistencia	SharedPreferences con JSON y conjuntos de cadenas. Está dentro de los ViewModel.
Actividad única	MainActivity crea los ViewModel e instala la interfaz con setContent.

Recorrido de una acción

Usuario toca un botón → onClick llama a un método → el ViewModel valida y actualiza MutableStateFlow → collectAsState observa el nuevo estado → Compose vuelve a dibujar lo que cambió. Si la acción persiste información, el ViewModel también escribe en SharedPreferences.

Ejemplo concreto

Al pulsar Me interesa, toggleInterest() agrega o quita el ID del trabajo en interested. La propiedad jobs calcula el total de votos. persist() guarda el conjunto. La pantalla recibe el nuevo ExploreState y actualiza el botón y el contador.

Qué mejorar al crecer

Extraer almacenamiento a repositorios; unificar los modelos Job y Publication; usar rutas tipadas y navegación estructurada; separar pantallas largas; introducir una identidad autenticada y reglas de autorización en el servidor. Estas son mejoras propuestas, no componentes ya presentes.

3. Carpetas y tecnologías

Ruta o tecnología	Para qué se usa
MainActivity.kt	Inicialización y selección de flujo principal.
core/component	PasswordVisibilityButton y espaciado/mapas adaptables.
core/theme	ServiNowTheme: colores Material 3 blanco y azul.
core/util	EmailValidator compartido.
features/login	Pantalla, estado y lógica del login.
features/register	Formulario, contraseña y AddressMapScreen.
features/recovery	Pantallas 05-07, RecoveryStep y cabecera compartida.
features/explore	Feed, filtros, mapas, detalle, comentarios y gestión de publicaciones.
assets/map	Leaflet 1.9.4: JavaScript, CSS y licencia incluidos en la app.
res/drawable	Iconos vectoriales XML: ojo, sobre, campana, casa, trabajo y otros.
src/test	Pruebas JUnit de reglas y cambios de estado.

Existe además ui/theme heredado de la plantilla. La actividad utiliza core/theme/ServiNowTheme; no debe confundirse con el tema anterior.

Configuración declarada: Kotlin Compose plugin 2.2.10; AGP 9.3.3; Compose BOM 2026.02.01; minSdk 26, compileSdk 37 y targetSdk 37; compatibilidad Java 11. Las versiones del catálogo describen la configuración, no un análisis completo de dependencias transitivas resueltas.

No se usan actualmente Firebase, Supabase, Room, Retrofit, Hilt ni Navigation Compose. Los mapas integran HTML/JavaScript dentro de WebView; la interfaz principal sí está escrita en Kotlin/Compose.

4. Estado, variables y conceptos Kotlin

Concepto	Explicación aplicada
val / var	val no permite reasignar la referencia; var sí. La UI usa estados inmutables y copy para generar una nueva versión.
data class	Agrupa datos. Ejemplos: LoginUiState, Publication, SearchFilters.
String? / Double?	El signo ? permite null: mensaje ausente o coordenada aún no seleccionada.
MutableStateFlow	Contenedor observable que el ViewModel puede modificar. _uiState o _state es privado.
asStateFlow()	Expone una vista de solo lectura del flujo para que la UI no lo cambie directamente.
update { it.copy(...) }	Transforma el estado anterior; it es ese estado. copy conserva los campos no indicados.
collectAsState()	Convierte el flujo en estado observado por Compose y dispara recomposición.
remember	Conserva valores entre recomposiciones, como un menú abierto; no es una base de datos.
rememberSaveable	En MainActivity conserva la ruta principal con el mecanismo de estado guardado de Android.
LaunchedEffect	Ejecuta una tarea al cambiar una clave: reiniciar el scroll cuando cambia page.
produceState / withContext	Cargan miniaturas fuera del hilo principal usando Dispatchers.IO.

ViewModel conserva el estado ante recreaciones de la actividad, pero no equivale a guardarlo tras la muerte del proceso. No se utiliza SavedStateHandle para restaurar todos los formularios; los borradores y contraseñas no tienen persistencia de larga duración.

5. Navegación y adaptación al celular

MainActivity mantiene screen con valores login, register, recovery y explore. Un when selecciona la función Route. Cada Route observa su ViewModel, muestra diálogos y organiza las pantallas. En exploración, ExploreState.page decide la subvista.

Variable / función	Papel
screen	Ruta principal; inicia en login.
page / previousPage	Subpantalla y origen del detalle para poder volver.
RecoveryStep	Enum EMAIL, SENT y NEW_PASSWORD. Evita confundir las tres etapas de recuperación.
choosingLocation	Decide si el registro muestra formulario o mapa.
BackHandler	Intercepta Atrás de Android y cambia la pantalla según el flujo.
Scaffold	Organiza área de contenido y barra inferior.
verticalScroll + rememberScrollState	Permite recorrer contenido mayor que el espacio disponible.
fillMaxWidth / weight	Reparte ancho y permite adaptar filas y botones.
imePadding / consumeWindowInsets	Ajusta espacio disponible al teclado y evita volver a aplicar determinados márgenes del sistema.
screenSpacing / mapHeight	Márgenes de 16 dp bajo 360 dp de ancho; 24 dp en otros casos. Mapa de 42% de alto, limitado a 220-380 dp.

La navegación usa cadenas y reglas manuales, no una pila completa de Navigation Compose. Por ejemplo, el regreso desde post-location conduce a create-details incluso si se abrió desde editar; es una limitación de navegación pendiente de pulir.

dp expresa medidas de interfaz independientes de densidad; sp se usa para texto. Los iconos son vectores y el contenido usa contenedores desplazables. No se ha certificado cada tamaño, orientación y escala de fuente en un dispositivo real.

6. Login y correos permitidos

LoginUiState	Significado
email / password	Texto introducido. Permanece en memoria del ViewModel.
passwordVisible	Decide si se usa PasswordVisualTransformation o texto visible.
rememberMe	Estado del checkbox. No conserva una sesión ni credenciales.
submitted	Activa las comprobaciones visuales tras intentar entrar.
message	Texto del AlertDialog; null significa que no hay diálogo.
emailError / passwordError	Propiedades calculadas: formato/dominio del correo y contraseña no vacía.

login() marca submitted, recorta espacios exteriores del correo y obtiene el primer error. Si todo es válido, informa que falta conectar autenticación. No consulta una cuenta ni compara contraseñas almacenadas.

onCredentialsRejected() prepara el mensaje “La contraseña es incorrecta”. Aún no hay proveedor que invoque ese rechazo. loginWithGoogle() muestra que la integración está pendiente; no solicita la contraseña de Google.

EmailValidator.error(value)

Devuelve String?; null equivale a correo aceptado por las reglas locales. Usa trim(), divide por @, valida la parte local y compara el dominio con una lista exacta, ignorando mayúsculas en el dominio.

Restricción	Implementación
Proveedores	gmail.com, hotmail.com, outlook.com y yahoo.com.
Formato	Exactamente una @; parte local entre 1 y 64 caracteres; total máximo 254.
Puntos	No permite punto inicial/final ni dos puntos consecutivos en la parte local.
Caracteres	Regex localPattern restringe el conjunto permitido.
Límite real	No confirma que la dirección exista ni que pertenezca a la persona. No admite variantes regionales no incluidas.

Ejemplo aceptado: persona@gmail.com. Rechazado: persona@gmail.com.ejemplo.com, porque el dominio no coincide exactamente.

7. Registro y reglas de contraseña

RegisterUiState añade name, confirmation, confirmationVisible, choosingLocation, latitude, longitude y locationConfirmed. submit() exige nombre no vacío, correo permitido, complejidad de contraseña y coincidencia exacta de la confirmación. Si pasa, activa choosingLocation.

Regla	Cómo se comprueba
Mínimo 8 caracteres	password.length < 8 produce error. Se habla de caracteres, no de ocho dígitos.
Una mayúscula	Debe existir al menos un carácter para el que isUpperCase() sea verdadero.
Un número	Debe existir al menos un carácter para el que isDigit() sea verdadero.
Un carácter especial	Debe existir uno que no sea letra, número ni espacio en blanco.
Confirmación	Comparación exacta password != confirmation.
Momento del aviso	Se muestra un AlertDialog al enviar; no se presenta un listado permanente de reglas.

Ejemplo didáctico válido: Casa2026! (no debe usarse como contraseña real). casa2026! falla por no tener mayúscula; Casa2026 falla por no tener carácter especial.

El código no obliga a una minúscula, no establece un máximo y no prohíbe todos los espacios: un espacio no satisface el carácter especial, pero puede existir junto con otro carácter especial válido. La longitud de String usa unidades UTF-16; no debe describirse como conteo avanzado de caracteres visuales.

El ojo no cifra la contraseña

PasswordVisibilityButton cambia la visibilidad mediante un booleano; PasswordVisualTransformation solo oculta lo que se dibuja. No hay hashing, credencial persistida ni servicio de autenticación. Las contraseñas no se escriben en SharedPreferences en este código.

selectLocation() acepta números finitos dentro de latitud [-90,90] y longitud [-180,180]. confirmLocation() exige un punto elegido. Los datos del registro y su dirección no se guardan como cuenta.

8. Recuperación: vistas 05, 06 y 07

Paso	Acción y método
05: EMAIL	Recupera tu acceso. emailChanged() actualiza el correo; submit() usa EmailValidator y pasa a SENT si es válido.
06: SENT	Revisa tu correo. El aviso explica que el prototipo no envía mensajes. resend() muestra un diálogo informativo.
Abrir enlace de prueba	previewLink() solo avanza si el estado es SENT. Es navegación de demostración, no validación de un enlace de correo.
07: NEW_PASSWORD	passwordChanged() y confirmationChanged() actualizan campos. savePassword() valida complejidad y coincidencia, pero no guarda una nueva credencial.
Regresar / reiniciar	back() retrocede y limpia ambas contraseñas; restart() vuelve a EMAIL conservando el correo.

La misma regla RegistrationPasswordValidator también se reutiliza en nueva contraseña, aunque su comentario de código dice “exclusiva del registro”. En una exposición conviene explicar el uso efectivo y no repetir ese comentario como si fuera una restricción técnica.

Qué falta para la recuperación real

Un proveedor debe enviar un enlace de un solo uso, comprobar su validez y expiración y efectuar el cambio de contraseña. La pantalla de prueba no debe habilitarse como vía de cambio real sin esa comprobación. Actualmente no existe enlace profundo de recuperación registrado en el manifest.

Los iconos de sobre y confirmación son drawables vectoriales sobre un Surface azul claro. Su contenedor está centrado horizontalmente. La campana solo muestra un aviso, no una bandeja real de notificaciones.

9. Feed, filtros y distancias

Modelo / campo	Uso
Job	id, title, category, area, price, minutes, votes, latitude, longitude y open.
exampleJobs	Tres trabajos escritos en código, con IDs enteros 1, 2 y 3.
SearchFilters	category, area, order, openOnly, latitude y longitude del centro de búsqueda.
ExploreState	page, query, filters, draft, selectedId, message, interested, comments, commentDraft y previousPage.
jobs / results	jobs suma el voto local al total base; results aplica filtros al listado derivado.

filterJobs() combina categoría, apertura, radio y texto. La búsqueda compara título, categoría y zona sin distinguir mayúsculas, pero no normaliza acentos. distanceKm() usa la fórmula de Haversine con radio terrestre 6371 km; entrega distancia aproximada en línea recta, no distancia por carretera.

Las opciones cercanas y otra ubicación usan hasta 5 km. “En mi ciudad · Bogotá” omite ese límite: con los ejemplos todos están en Bogotá, pero no hay validación geográfica de municipio. El centro inicial es fijo; “cerca de mí” no obtiene GPS.

Orden: minutos ascendentes para recientes; votos descendentes para relevancia; distancia ascendente para cercanía. Los minutos son valores de ejemplo y no un reloj en vivo.

Métodos principales

query() y category() cambian resultados directamente. editFilters() copia filtros a draft. draft() modifica esa copia. apply() confirma y regresa al feed. reset() restablece el borrador y limpia la búsqueda; los filtros se confirman con Aplicar. select() cambia el marcador elegido; openJob() abre un ID existente.

10. Cómo se integró el mapa

El mapa utiliza Leaflet 1.9.4 y teselas de OpenStreetMap. No usa el SDK de Google Maps ni Mapbox y no requiere una clave de Google. AddressMapScreen sirve al registro; ExploreMap sirve al feed, filtros, detalle y ubicación de publicaciones.

Componente	Funcionamiento
AndroidView	Inserta una vista Android clásica dentro de Compose.
WebView	Renderiza un pequeño documento HTML con JavaScript y CSS. JavaScript se habilita explícitamente.
assets/map	Contiene leaflet.js, leaflet.css y la licencia, empaquetados con la app.
loadDataWithBaseURL	Carga el HTML con origen local lógico https://appassets.androidplatform.net/map/.
shouldInterceptRequest	Sirve los archivos de Leaflet desde assets para ese host. No hay servidor web de la app.
L.map / setView	Crea el mapa y lo centra en las coordenadas iniciales.
L.tileLayer	Solicita imágenes del mapa a https://tile.openstreetmap.org/{z}/{x}/{y}.png.
L.marker / circleMarker	Dibuja precios de trabajos o el punto seleccionado por el usuario.

Del toque en el mapa a una variable Kotlin

Leaflet captura click → obtiene e.latlng → genera una URL servinow://point?lat=...&lng=... (location en registro) → shouldOverrideUrlLoading intercepta la URL → convierte los parámetros a Double → valida finitud/rango → ejecuta el callback onLocation u onSelect → actualiza el ViewModel.

servinow://job?id=... selecciona un trabajo existente. Estos son mensajes internos interceptados; no hay un puente addJavascriptInterface expuesto. Los enlaces permitidos de atribución de OpenStreetMap se abren externamente.

11. Mapa: permisos y límites

AndroidManifest.xml declara INTERNET. No solicita ACCESS_FINE_LOCATION ni ACCESS_COARSE_LOCATION, porque la dirección se elige tocando el mapa. El punto inicial de Bogotá es una referencia visual, no la posición detectada del usuario.

No hay buscador geográfico ni conversión inversa de coordenadas a calle/dirección postal. Elegir un punto entrega latitud/longitud. Las etiquetas de barrios del feed son datos de ejemplo, no resultados de geocodificación.

El acceso de WebView a archivos y contenido está desactivado; los assets se suministran mediante el interceptor. Se conserva la atribución a OpenStreetMap, se añade ServiNow al User-Agent y las teselas usan HTTPS. onRelease detiene y destruye el WebView.

Conexión y actualización

Leaflet está incluido localmente, pero las teselas necesitan internet; puede haber caché del WebView. No existe descarga de mapas sin conexión. Se presenta un aviso cuando una tesela falla; en registro hay Recargar mapa. No hay una prueba integral registrada de todos los gestos de mapa en cada pantalla.

ExploreMap construye su HTML en factory y no tiene bloque update para refrescarlo ante cualquier cambio de parámetros. Los callbacks sí usan rememberUpdatedState. Al evolucionar el flujo debe revisarse la actualización del mapa cuando cambia el conjunto de trabajos sin recrear la vista.

Cómo explicarlo en clase

“La pantalla está hecha en Compose, pero para el mapa reutilizamos Leaflet dentro de WebView. Kotlin recibe coordenadas mediante un esquema de URL interceptado. OpenStreetMap aporta las imágenes geográficas. No estamos usando GPS ni un servicio que transforme coordenadas en una dirección escrita”.

12. Fotos: selección, acceso y vista previa

Actualmente no hay subida de imágenes a internet. La app abre el selector de documentos de Android y conserva referencias URI. Una URI content:// identifica contenido accesible mediante un proveedor del sistema; no es una URL pública ni necesariamente una ruta de archivo.

Elemento	Qué hace
rememberLauncherForActivityResult	Registra la respuesta de la actividad externa del selector.
OpenMultipleDocuments	Permite elegir varios documentos; se solicitan image/jpeg e image/png.
takePersistableUriPermission	Intenta conservar permiso de lectura para volver a acceder después.
photos: List<String>	Guarda las URI aceptadas, convertidas a texto y sin duplicados.
LocalPhoto(uri)	Abre el contenido con ContentResolver y lo decodifica para mostrarlo.
BitmapFactory.Options	Primero lee dimensiones sin crear el bitmap completo.
inSampleSize	Aumenta la reducción hasta que las dimensiones decodificadas queden alrededor de 1200 px o menos.
produceState / Dispatchers.IO	Realiza lectura y decodificación fuera del hilo de UI.
Image / ContentScale.Crop	Dibuja la vista previa en un marco redondeado de 200 dp de alto.

Quitar foto elimina su referencia del borrador; no borra el original del dispositivo. Si el proveedor revoca acceso o el documento desaparece, la vista previa puede fallar. La app muestra un texto de ausencia de vista previa.

No hay límite explícito de número o peso de fotos ni verificación profunda de archivo; el filtro MIME y la decodificación no son un sistema completo de validación de subida. El permiso persistente tampoco copia el archivo a almacenamiento propio.

13. Qué falta para subir imágenes

El requisito de servicio externo de imágenes está pendiente. No se ha configurado Cloudinary, S3, Google Cloud Storage ni otro proveedor. Guardar URI en SharedPreferences no cumple todavía la subida remota.

Flujo futuro propuesto, no implementado

1. Seleccionar la foto y comprobar tamaño, formato y contenido decodificable. 2. Generar una versión adecuada para envío. 3. Subirla mediante el servicio autorizado. 4. Recibir una URL o identificador remoto. 5. Guardarlo junto con la publicación en la base de datos. 6. Mostrar progreso, reintento y errores sin perder el borrador.

Diferencias que debes poder explicar

Ahora	Con un servicio remoto
URI local content://	URL o clave del archivo remoto.
Accesible en el dispositivo con permiso	Acceso entre dispositivos según reglas del servicio.
Solo se guarda la referencia local	El archivo se transfiere al almacenamiento externo.
Sin indicador de subida	Estado de carga, éxito y error.
Quitar referencia del borrador	Política explícita para conservar o borrar archivos remotos.

La aplicación no debe llevar credenciales administrativas del servicio en el APK. El mecanismo concreto dependerá del proveedor elegido y debe diseñarse con permisos limitados. Este documento no afirma que esa integración ya exista.

14. Crear, editar y gestionar publicaciones

Modelo Publication	Descripción
id: String	UUID generado al crear el borrador; permite sustituir la misma publicación al editar.
title / category / description	Información textual; categoría inicial Plomería.
budget: String	Se conserva como texto del formulario; se valida como Long positivo.
photos: List<String>	URI de fotos seleccionadas.
latitude / longitude: Double?	null hasta elegir ubicación; después coordenadas válidas.
status / reason	Estado y motivo de rechazo. Por defecto Pendiente y motivo vacío.
PublicationState	posts, draft, editing, filter y message controlan lista y formulario.

13: validate(1) comprueba título y categoría. 14: validate(2) añade descripción, presupuesto entero positivo y al menos una foto. 15: validate(3) añade coordenadas finitas dentro de rango. 16: save() vuelve a validar todo antes de persistir. 17: comunica el estado pendiente.

save() recorta el título, impone status = Pendiente, borra reason y sustituye cualquier registro con el mismo ID. Si el guardado falla, no cambia la lista y muestra error. No envía nada a un moderador real.

18: filter selecciona Todas, Activas (status Verificada), Pendientes o Finalizadas. 19: edit(post) copia el registro al borrador; change() actualiza sus campos; se pueden cambiar fotos y ubicación. delete(post) se ejecuta tras un diálogo de confirmación.

20: si el borrador es Rechazada se muestra su motivo; de lo contrario se presenta un ejemplo explícito. No hay una función que asigne rechazos reales. “Crear publicación corregida” inicia un borrador nuevo en el modo de ejemplo.

15. Persistencia y separación de datos

Almacén	Contenido y mecanismo
explore_local	interested: conjunto de IDs como texto. comments: objeto JSON que asocia ID de trabajo a lista de comentarios. Escritura con apply().
publications_local	posts: cadena JSON con arreglo de publicaciones. read() reconstruye objetos; persist() usa commit() y devuelve resultado.
Memoria del ViewModel	Contraseñas, formularios de acceso, ubicación del registro, filtros y borradores todavía no guardados.
Código fuente	exampleJobs, categorías y textos de demostración.

SharedPreferences es almacenamiento de pares clave-valor privado de la app, no una base de datos relacional ni un servidor. El JSON es una representación textual de listas y objetos. No hay sincronización entre usuarios ni reglas de autorización por cuenta.

Los comentarios pertenecen a los IDs enteros de los trabajos de ejemplo; las publicaciones propias utilizan UUID y otro modelo. Por eso crear una publicación no la inserta automáticamente en exampleJobs y los comentarios no están integrados todavía con las publicaciones propias.

La persistencia de publicaciones ocurre al guardar, no en cada pulsación: el borrador puede perderse al terminar el proceso. commit() se usa de forma síncrona desde el flujo de UI; para crecer conviene mover esas escrituras a IO y usar repositorios o una base local.

Los errores de lectura JSON se convierten en listas vacías mediante runCatching. No hay migraciones de esquema ni recuperación avanzada de corrupción. El manifest permite backup, pero no se ha validado una estrategia de restauración de fotos y permisos.

16. Comentarios e interés

interested es un Set<Int>: cada ID aparece como máximo una vez. toggleInterest() lo agrega si no estaba o lo quita si estaba. El voto local se suma a los votos base del trabajo sin modificar exampleJobs.

comments es Map<Int, List<String>>. La clave identifica el trabajo y los textos forman su conversación local. No se guardan autor autenticado, fecha ni ID de comentario. El nombre “Tú” es una etiqueta visual.

postComment() paso a paso

Lee selectedId y commentDraft; aplica trim(); rechaza vacío y texto de más de 1000 caracteres; añade el comentario a la lista del ID correspondiente; limpia commentDraft; llama a persist(). El usuario ve el cambio mediante StateFlow.

La pantalla 12 presenta los comentarios por separado y el detalle también conserva un bloque de comentarios. La navegación “Abrir conversación” lleva a comments; Atrás devuelve a detail.

Diferencia frente al requisito completo

No se crea una notificación para el autor ni se publica el comentario en un servidor. Estos datos no tienen identidad multiusuario. Para cumplir el requisito deben persistirse comentario y notificación con el destinatario, publicación, fecha y estado leído/no leído, y aplicarse permisos en el backend.

17. Métodos: cómo seguir el código

Clase / métodos	Responsabilidad
MainActivity.onCreate	Crea ViewModel, adjunta almacenamiento y prepara Compose.
RegisterViewModel.submit / selectLocation / confirmLocation	Valida registro, selecciona punto y confirma ubicación temporal.
RecoveryViewModel.submit / previewLink / savePassword	Transiciones de demostración y validación de nueva contraseña.
ExploreViewModel.attachStorage / persist	Carga y guarda intereses y comentarios locales.
ExploreViewModel.openJob / back / page	Selecciona un trabajo y resuelve cambios de vista.
ExploreViewModel.editFilters / draft / apply / reset	Edición temporal y aplicación de filtros.
ExploreViewModel.toggleInterest / postComment	Modifica participación local y la persiste.
PublicationViewModel.start / edit / change	Inicia o modifica el borrador de publicación.
PublicationViewModel.validate / save / delete	Valida, guarda o elimina publicaciones.
publicationError / filterJobs / distanceKm	Funciones de reglas sin interfaz; aptas para pruebas unitarias.
LocalPhoto / ExploreMap	Funciones de UI con carga de fotos e integración del mapa.

Cómo depurar una acción

Si un botón no hace lo esperado, localiza su onClick en la pantalla, identifica el método llamado, inspecciona el estado antes y después y verifica el dato persistido. No basta con que el botón se vea: debe modificar estado, navegar o ejecutar una operación verificable.

18. Restricciones y trabajo pendiente

Área	Límite comprobado en el código
Seguridad de acceso	No hay cuentas reales, sesiones, roles aplicados ni permisos de servidor. Google es un botón pendiente.
Publicaciones	Título y descripción solo se comprueban no vacíos; no hay límites máximos definidos. Presupuesto debe ser Long positivo.
Fotos	Mínimo una referencia; sin subida externa, cuota de archivos, validación avanzada o recuperación de permisos perdidos.
Mapas	Sin GPS ni geocodificación. Internet para teselas. La búsqueda por ciudad no comprueba límites administrativos.
Feed propio	Las nuevas publicaciones quedan en la lista local; no hay moderación ni actualización remota del feed.
Rechazo	Vista ilustrativa o lectura de status Rechazada; no hay actor moderador que lo genere.
Navegación	Rutas manuales, algunos retornos genéricos y botones de otras secciones aún informativos.
Datos	Dos modelos de publicación separados, JSON sin migraciones y datos sin separación por usuario.
Resto del alcance	Ofertas, CV/perfiles, calificaciones, finalización completa, notificaciones, métricas, niveles, insignias y repositorio colaborativo no acreditados como terminados.

No conviene presentar el prototipo como “backend terminado” o “subida de imágenes implementada”. La formulación precisa es: interfaz Android y operaciones locales funcionales, con integraciones remotas pendientes.

19. Pruebas y verificación

Suite	Pruebas / fallos / errores
ExampleUnitTest	1 / 0 / 0
EmailValidatorTest	3 / 0 / 0
ExploreTest	6 / 0 / 0
PublicationTest	2 / 0 / 0
LoginViewModelTest	5 / 0 / 0
RecoveryViewModelTest	4 / 0 / 0
RegisterViewModelTest	6 / 0 / 0

Estos resultados proceden de los reportes Gradle existentes de la última ejecución. No se volvieron a ejecutar para generar este documento. La compilación assembleDebug y las pruebas unitarias habían concluido correctamente en la última modificación de las vistas hasta la 20.

Cobertura: correo, complejidad de contraseña, estados del login y registro, recuperación 05-07, filtros, distancia, interés reversible, comentarios y campos obligatorios de publicación. Hay una prueba heredada de la plantilla en el paquete prueba; no debe contarse como prueba funcional de ServiNow.

Verificación manual registrada: el feed se abrió y se desplazó en emulator-5554. Después el emulador dejó de estar conectado; no se completó una prueba visual e integral de las vistas recién añadidas 13-20, permisos de fotos, persistencia tras reinicio y mapas en todas sus variantes.

Comandos para repetir

Desde la carpeta del proyecto: ./gradlew :app:assembleDebug :app:testDebugUnitTest --offline --no-configuration-cache. El proyecto se compiló usando JAVA_HOME=/home/fafa/android-studio/jbr. --offline presupone dependencias ya descargadas.

Prueba manual recomendada

Abrir demostración → buscar y filtrar → detalle → votar/comentar → crear → intentar avanzar sin foto → seleccionar foto → elegir punto → guardar → cerrar/abrir → editar → eliminar con confirmación. Comprobar que la nueva publicación no aparece como verificada en el feed.

20. Guion para explicar el proyecto

Presentación breve

“ServiNow conecta solicitudes de servicios con trabajadores. La versión actual está hecha en Kotlin y Compose. Organicé el código por funciones, con ViewModel y StateFlow para separar la interfaz de sus cambios de estado. Tiene validación de acceso, recuperación de demostración, búsqueda, mapas, comentarios y gestión local de publicaciones”.

Demostración en orden

1. Mostrar el registro: error de correo o contraseña y posterior paso al mapa. 2. Mostrar 05 → 06 → 07 aclarando que no se envía correo. 3. Entrar a explorar y filtrar. 4. Abrir detalle, cambiar interés y comentar. 5. Crear una publicación con foto y coordenadas. 6. Ver pendiente en Mis publicaciones y editarla.

Preguntas que pueden hacerte

Pregunta	Respuesta breve
¿Es MVVM?	Sí, un enfoque MVVM básico; la persistencia aún está dentro del ViewModel, sin repositorios.
¿Dónde está la base de datos?	No hay remota; hay SharedPreferences con JSON para publicaciones, comentarios e intereses.
¿Se suben las imágenes?	No todavía. Selecciono documentos, conservo URI/permisos y muestro miniaturas.
¿El mapa es Google?	No. Leaflet en WebView con teselas OpenStreetMap y callbacks hacia Kotlin.
¿El ojo protege la contraseña?	Solo controla visibilidad; la autenticación y almacenamiento seguro del proveedor aún faltan.
¿Qué significa Pendiente?	Estado local inicial. No implica que un moderador real haya recibido la solicitud.
¿Qué diferencia una vista de una función?	La función cambia estado o datos verificables. Aquí comentarios, votos y CRUD local ya lo hacen.

21. Índice de archivos y funciones

Rutas relativas a app/src/main/java/com/example/servinow. Este inventario ayuda a buscar nombres en Android Studio; las páginas previas explican su propósito. Las funciones composables dibujan UI; las funciones de ViewModel gestionan acciones y estado.

MainActivity.kt

onCreate

core/component/AdaptiveSpacing.kt

screenSpacing, mapHeight

core/component/PasswordVisibilityButton.kt

PasswordVisibilityButton

core/theme/ServiNowTheme.kt

ServiNowTheme

core/util/EmailValidator.kt

error

features/explore/ExploreMap.kt

ExploreMap, shouldInterceptRequest, shouldOverrideUrlLoading

features/explore/ExploreScreen.kt

ExploreRoute, Heading, Primary, Choice, JobCard

features/explore/ExploreViewModel.kt

distanceKm, filterJobs, attachStorage, persist, openJob, back, toggleInterest, comment, postComment, page, query, category, editFilters, draft, apply, reset, select, message

features/explore/PublicationScreens.kt

publicationTitle, PublicationContent, Step, Title, Action, Field, LocalPhoto

features/explore/PublicationViewModel.kt

read, persist, start, edit, change, filter, message, validate, save, delete, publicationError

features/login/LoginScreen.kt

LoginRoute, LoginScreen, LoginPreview

features/login/LoginViewModel.kt

onEmailChange, onPasswordChange, togglePassword, rememberMeChanged, dismissMessage, showPendingFeature, loginWithGoogle, onCredentialsRejected, login

features/recovery/NewPasswordScreen.kt

NewPasswordScreen, PasswordInput, NewPasswordPreview, RecoveryHeader

features/recovery/RecoveryScreen.kt

RecoveryRoute, RecoveryStepsScreen, Recovery05Preview, Recovery06Preview

features/recovery/RecoveryViewModel.kt

restart, back, previewLink, resend, emailChanged, dismissMessage, passwordChanged, confirmationChanged, notifications, savePassword, submit

features/register/AddressMapScreen.kt

AddressMapScreen, shouldInterceptRequest, shouldOverrideUrlLoading, mapHtml

features/register/RegisterScreen.kt

RegisterRoute, RegistrationPasswordField

features/register/RegisterViewModel.kt

nameChanged, emailChanged, passwordChanged, confirmationChanged, togglePassword, toggleConfirmation, dismissMessage, google, backToForm, selectLocation, confirmLocation, submit

features/register/RegistrationPasswordValidator.kt

error

22. Fuentes y alcance del informe

Fuente principal: inspección directa del proyecto ServiNow y de los reportes de pruebas presentes. El estado funcional se contrastó con los mockups numerados de work/screens.json. No se modificó la aplicación para producir este informe.

Archivos de referencia

MainActivity.kt; core/util/EmailValidator.kt; core/component/AdaptiveSpacing.kt y PasswordVisibilityButton.kt; todos los archivos de features/login, features/register, features/recovery y features/explore; AndroidManifest.xml; app/build.gradle.kts; gradle/libs.versions.toml; assets/map; app/src/test y app/build/test-results/testDebugUnitTest.

Material para continuar estudiando

El material Kotlin.pdf aportado por el grupo es una guía educativa. Para estudiar las APIs usadas, consulta la documentación oficial de Android sobre Compose, ViewModel, StateFlow, Activity Result APIs, Storage Access Framework y WebView. Leaflet incluye su licencia en assets/map/LICENSE. Las condiciones del proveedor de teselas deben revisarse antes de publicar una aplicación a escala.

Lista de prioridades para la siguiente etapa

Conectar proveedor de autenticación y recuperación real; definir identidad y roles; separar repositorios; definir base remota y reglas; implementar subida de imágenes; unificar Job y Publication; implementar moderación y notificaciones; añadir ofertas/perfiles; probar flujos completos en emulador y teléfono; configurar repositorio GitHub y contribuciones del grupo.

La versión entregada es una base de desarrollo local. Su valor actual está en los flujos implementados y verificables; las limitaciones anteriores forman parte de la explicación técnica y no deben ocultarse en la presentación.